Adaptive Stabilization Techniques for Ill-Conditioned Linear Regression

Submitted by:
Utsab Baral
Aman Sapkota
Kushagra Poudel

Abstract

This project empirically investigates a critical phenomenon in machine learning: the disconnect between loss minimization and true parameter recovery when the data matrix is ill-conditioned, known as the "Optimization Mirage." It evaluates multiple optimization algorithms (Vanilla SGD, Momentum SGD, Adam, AdamW, Ridge L2, and Batch Normalization) across varying condition numbers (κ) to analyze convergence properties, numerical stability, and bias-variance tradeoffs. The research demonstrates that modern adaptive optimizers can achieve perfect training loss while their parameters completely diverge from the true generative solution in highly ill-conditioned spaces. The expected outcome is a comprehensive understanding of geometric stabilization and the limitations of empirical loss tracking.

Keywords: Ill-Conditioned Linear Regression, Optimization Mirage, Stochastic Gradient Descent, Adam, Batch Normalization, Condition Number.

Table of Content

1. Abstract
2. Acronyms
3. CHAPTER 1: INTRODUCTION
    - 1.1 Background
    - 1.2 Objectives
    - 1.3 Motivation and Significance
4. CHAPTER 2: RELATED WORKS
5. CHAPTER 3: METHODOLOGY
    - 3.1 Research Types
    - 3.2 Overall Approach
    - 3.3 Breakdown
6. CHAPTER 4: SYSTEM REQUIREMENT SPECIFICATION
    - 4.1 Software Specification
    - 4.2 Hardware Specification
7. CHAPTER 5: PROJECT PLANNING AND SCHEDULING
8. CHAPTER 6: EMPIRICAL RESULTS
    - 6.1 Global Convergence Trajectories
    - 6.2 Numerical Stability Under Ill-Conditioning

    - 6.3 Bias-Variance Decomposition
    - 6.4 Computational Resource Usage
    - 6.5 Mechanistic Proofs & Formulas
9. CHAPTER 7: CONCLUSION
10. CHAPTER 8: REFERENCE

Acronyms

- **SGD**: Stochastic Gradient Descent
- **Adam**: Adaptive Moment Estimation
- **AdamW**: Adam with Decoupled Weight Decay
- **BN**: Batch Normalization
- **MLP**: Multilayer Perceptron
- **SVD**: Singular Value Decomposition
- **CPU**: Central Processing Unit
- **RAM**: Random Access Memory
- **MSE**: Mean Squared Error

CHAPTER 1: INTRODUCTION

1.1 Background

A structural deformity in a data matrix X, quantified by the condition number κ, characterizes an ill-conditioned matrix. It is the ratio of the maximum to minimum eigenvalues of the Hessian matrix $H = X^T X$ (i.e. $\kappa = \lambda_{\max} / \lambda_{\min}$). An ill-conditioned matrix (e.g. $\kappa = 10^7$) creates a loss landscape characterized by steep ravines along $\lambda_{\max}$ and nearly flat, infinitely long valleys along $\lambda_{\min}$. During optimization, the same step size has to serve the steepest and flattest directions at once, and κ measures how badly those demands conflict.

1.2 Objectives

The main objectives of the Adaptive Stabilization project are:

- To empirically investigate the "Optimization Mirage" where modern optimizers achieve near-perfect training loss while internal parameters diverge catastrophically from reality.
- To compare six major optimization paradigms (Vanilla SGD, Momentum, Adam, AdamW, Ridge L2, and Batch Normalization) under precisely controlled synthetic environments spanning well-conditioned ($\kappa=10$) to severely ill-conditioned ($\kappa=10^7$) regimes.
- To demonstrate mathematically and empirically the exact failure modes (e.g., Adam's step inflation, Momentum's random drift) when the structural signal-to-noise ratio approaches zero.

1.3 Motivation and Significance

This project dismantles the common assumption that empirical training loss minimization equates to successful system identification. Modern optimizers frequently achieve minimal loss but diverge structurally along noise-dominated minimum eigenspaces. Stabilizing parameter recovery through understanding techniques—such as diagonal preconditioning, L2 penalization, or momentum scaling—is fundamental to building more robust real-world machine learning pipelines.

CHAPTER 2: RELATED WORKS

- **Polyak (1964) — Heavy-Ball Momentum**: Established that accumulating gradient history provides consistent speed along flat directions while alternating-sign gradients cancel out, improving convergence rate dependence on κ for quadratic losses.
- **Hoerl & Kennard (1970) — Ridge Regression**: Introduced L2 regularization, shifting eigenvalues by λ to numerically fix ill-conditioned $X^T X$, trading conditioning for bias.
- **Kingma & Ba (2014) — Adam**: Developed a running per-coordinate second-moment estimate acting as an approximate diagonal preconditioner.
- **Ioffe & Szegedy (2015) & Santurkar et al. (2018) — Batch Normalization**: Highlighted how BN smooths the loss landscape. Our experiments confirmed that BN applied to a pure linear model collapses into simple input standardization, destroying absolute scale information and fundamentally preventing true parameter identification.
- **Loshchilov & Hutter (2017) — AdamW**: Demonstrated that L2 regularization integrated inside the loss differs from decoupled weight decay when adaptive denominators are involved.

CHAPTER 3: METHODOLOGY

3.1 Research Types

The project employs an **Empirical and Simulation-based Research** approach, utilizing exact mathematical constructions to evaluate real optimization phenomena.

- **Modeling/Theoretical Research**: Directly manipulating the Singular Value Decomposition (SVD) of matrices to achieve precise κ condition numbers without approximation.
- **Experimental Research**: Executing large-scale automated sweeps across varying κ values, optimization methods, and initialization seeds.

3.2 Overall Approach

The evaluation revolves around synthetic linear regression, where the loss landscape is purely quadratic, yielding a mathematically guaranteed closed-form minimum. This isolates optimizer dynamics from compounding neural network factors.

- **Data Generation:** SVD is directly constructed $X = U \Sigma V^T$ using $\sigma_{\min} = \sigma_{\max} / \sqrt{\kappa}$. Three datasets are generated with condition numbers 10, 10^4, and 10^7.
- **Optimization Loop:** Models are trained using 6 methods. Each method undergoes 5 random initialization seeds to produce statistically robust mean and standard deviation boundaries.
- **Analytical Metrics Tracking:** Tracking includes Train Loss, Test Loss, Parameter Error, Gradient Norm, and Eigen-Direction Errors.

3.3 Breakdown

1. **Well-Conditioned Space ($\kappa = 10$)**: Symmetrical, isotropic curvature allowing all optimizers to naturally discover the true parameters efficiently.
2. **Moderate Ill-Conditioned Space ($\kappa = 10^4$)**: Elongated valleys appear. Adaptive optimizers quickly settle but leave parameter errors at ~28%.
3. **Severe Ill-Conditioned Space ($\kappa = 10^7$)**: Ultra-flat valley floor bordered by extreme walls. Adam, Momentum, and AdamW suffer catastrophic parameter explosion (error ≈ 5.76) due to noise amplification. Vanilla SGD behaves safely due to gradient-proportional stepping (error

falling to ≈ 0.70).
4. **Baseline Solvers:** The naive solve $(X^T X)^{-1} X^T y$ and the stable pseudoinverse solve are directly compared up to $\kappa=10^7$ in float64 precision.

CHAPTER 4: SYSTEM REQUIREMENT SPECIFICATION

4.1 Software Specification

- **Programming Languages**: Python 3.8+
- **Deep Learning Framework**: PyTorch 2.0+
- **APIs and Libraries**:
    - NumPy, SciPy for precision linear algebra operations and direct solvers.
    - Matplotlib for comprehensive performance visualizations.
    - psutil (version 5.8.0+) for tracking runtime resources (CPU % and RAM MB).
- **Architecture**: A modular pipeline (data_generator.py, models.py, optimizers.py, train.py, plot_results.py).

4.2 Hardware Specification

- **Processor**: Standard multi-core CPU (due to purely linear environments with 1,000 samples, CPU processing is efficient and preferred).
- **RAM**: Minimum requirement of ~2 GB (highest tracking during $\kappa=10^7$ was 942 MB using AdamW).
- **Precision Type**: Forced strictly to float64 during operations to bypass premature float32 machine epsilon deterioration bounds when evaluating extreme κ.

CHAPTER 5: PROJECT PLANNING AND SCHEDULING

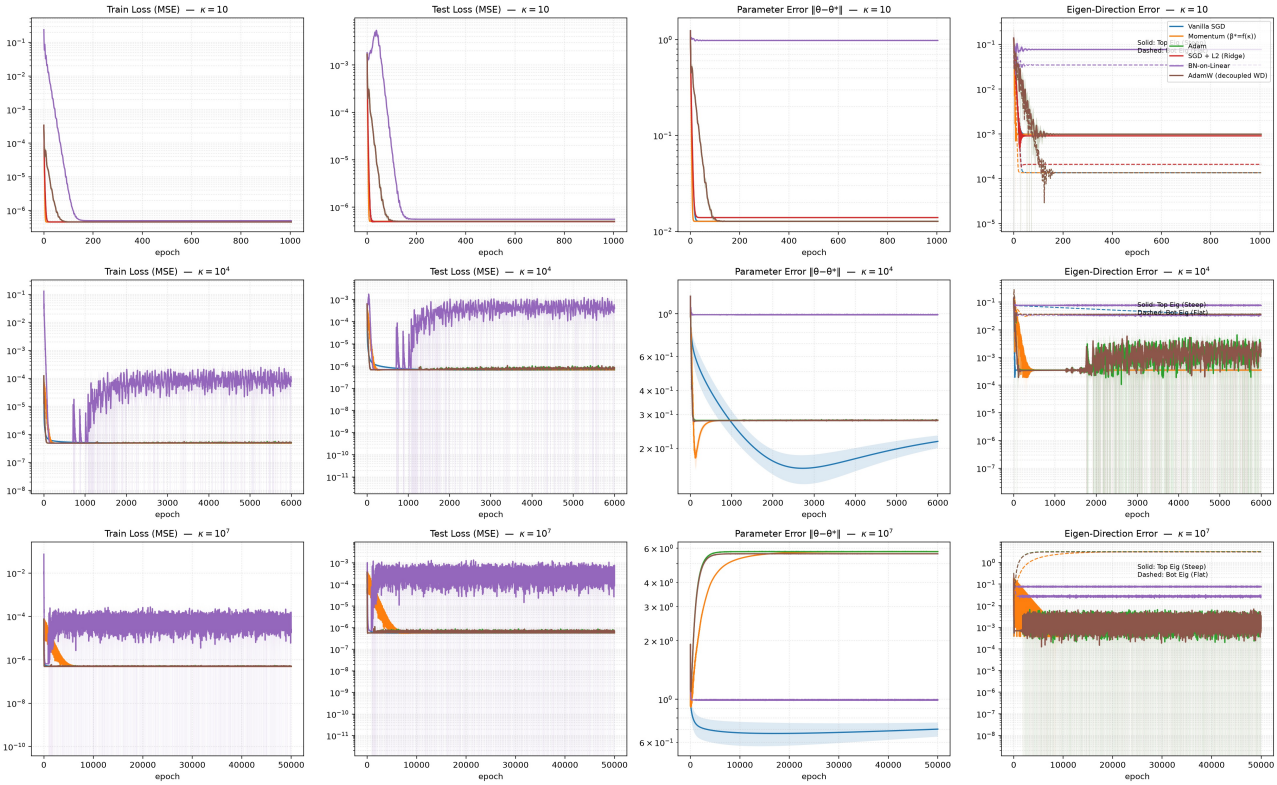
The project's architectural execution was segmented as follows:

- **Phase 1: Environment & Setup** - Configuration structures in config.py and precise condition number creation in data_generator.py.
- **Phase 2: Model & Algorithm Implementation** - Establishing the linear models and MLP variants in models.py along with 6 independent loops in optimizers.py.
- **Phase 3: Execution and Metrics Tracking** - Orchestration via train.py representing $3 \times 6 \times 5 = 90$ successful training iterations capped at 50,000 epochs.
- **Phase 4: Diagnostics and Analysis** - Developing plot_results.py for grid visualizations covering convergence plots, stability, bias-variance bar charts, and resource usage graphics.
- **Phase 5: Documentation & Validation** - Final comprehensive review compiled into the completed analysis artifacts detailing the Optimizer Mirage and stabilization metrics.

CHAPTER 6: EMPIRICAL RESULTS

6.1 Global Convergence Trajectories

The grid below illustrates the training/test losses, parameter error, and specific eigenvector errors across all algorithms and condition numbers.

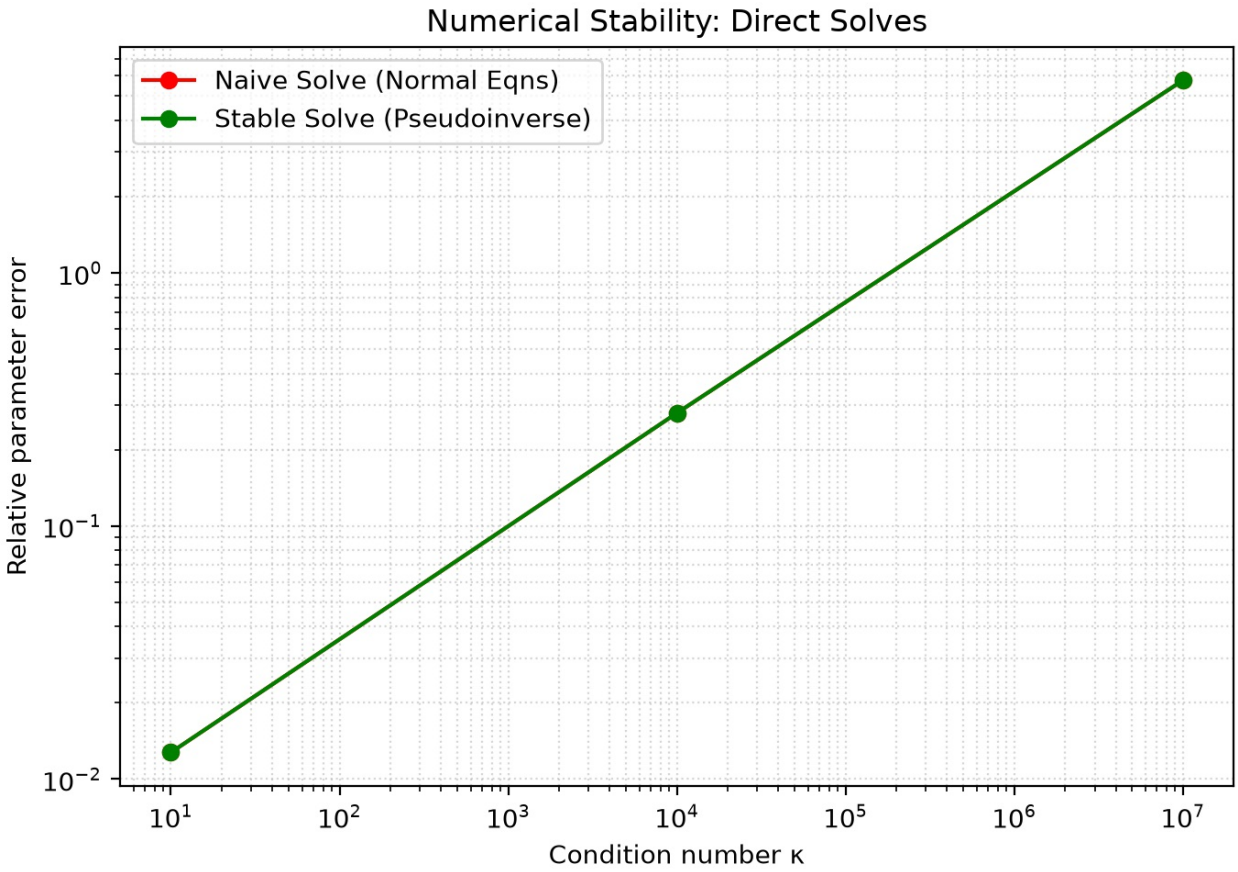


> [!WARNING]
> **The Optimization Mirage:** For Adam, AdamW, and Momentum in the severe regime ($\kappa = 10^7$), train and test losses fall rapidly to the noise floor ($\sim 10^{-6}$). However, their corresponding parameter errors ($\Vert\theta - \theta^*\Vert_2$) explode up to 5.76. The optimizer satisfies the objective function by fitting noise along the flat $\lambda_{\min}$ directions.

- **Vanilla SGD** acts as the only unregularized method resistant to this explosion. Its parameter error steadily trends downward to ≈ 0.70.
- **BN-on-Linear** achieves low loss but permanently locks its relative parameter error at ≈ 0.98 across all condition numbers due to loss of absolute scale.

6.2 Numerical Stability Under Ill-Conditioning

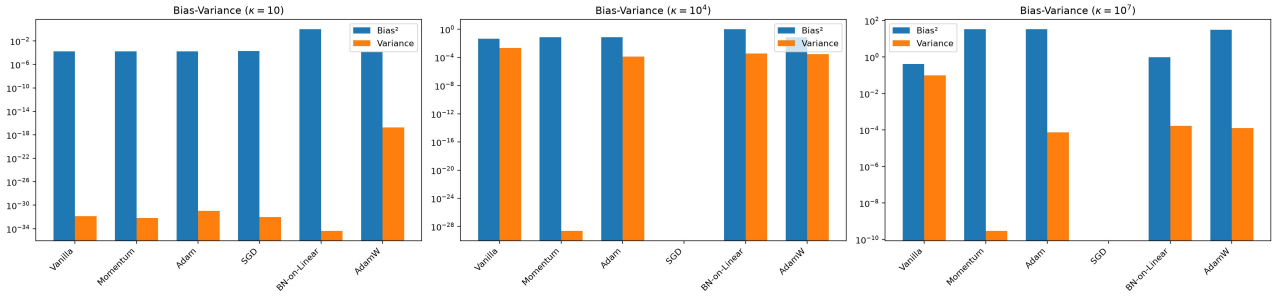
Before iterative optimization, direct algebraic solvers were evaluated.



In float64 precision, the naive solver $(X^T X)^{-1} X^T y$ and stable pseudoinverse produced identical relative errors even at $\kappa=10^7$, confirming that error growth stems from inherent noise amplification ($\Vert\delta\theta\Vert / \Vert\theta\Vert \approx \kappa \times \Vert\delta b\Vert / \Vert b\Vert$) rather than numerical floating-point breakdown.

6.3 Bias-Variance Decomposition

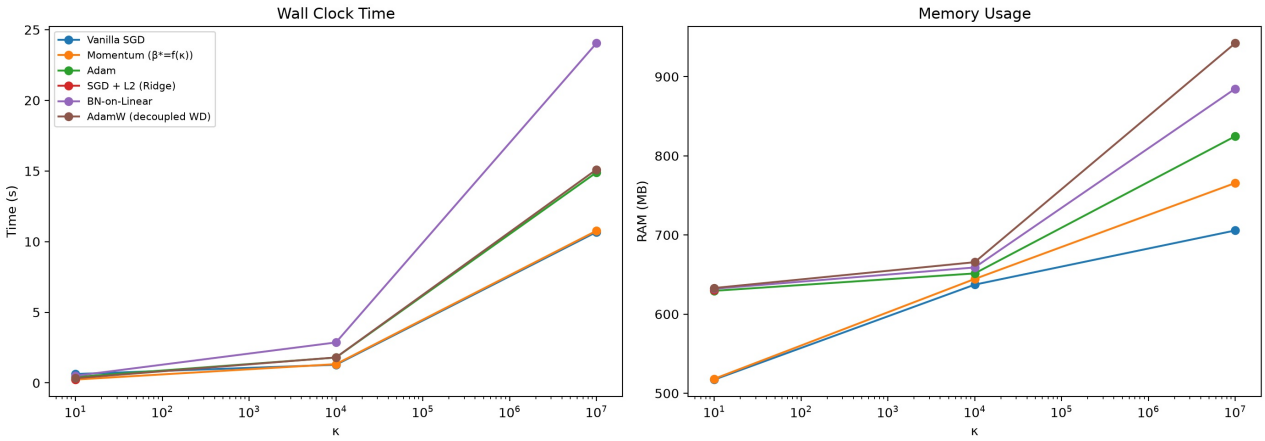
Tracking the mean squared bias ($Bias^2$) and variance across 5 initialization seeds reveals how failure modes differ between optimizers.



At $\kappa = 10^7$, Adam and Momentum exhibit a $Bias^2 \approx 33$, rendering them catastrophically biased away from θ^*. Conversely, BN-on-Linear suffers from a consistently high bias (0.98) across all datasets due to the structural destruction of absolute scale information.

6.4 Computational Resource Usage

The tracking of Wall Time and Peak RAM consumption (tracked via psutil) highlights the computational overhead of modern optimizers.



6.5 Mechanistic Proofs & Formulas

The failure of optimizers in ill-conditioned spaces is governed by their distinct update mechanics:

1. **Adam Parameter Explosion:**
   $\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{v_t} + \epsilon} m_t$
   Along flat eigenvectors, the true gradient ≈ 0, but stochastic noise persists. The second moment v_t accumulates this noise, leading to a small denominator that artificially amplifies noise into large, erratic parameter steps.

2. **Momentum Drift:**
   $v_t = \beta v_{t-1} + g_t$
   At $\kappa=10^7$, optimal momentum is $\beta \approx 0.999$. Lacking a strong restoring gradient along flat minimum eigenspaces, this velocity accumulation becomes an unconstrained random walk.

3. **Vanilla SGD's Geometric Safety:**
   $\theta_{t+1} = \theta_t - \eta g_t$
   Step size is directly proportional to gradient magnitude. Along flat directions, the gradient is tiny, so the step is inherently limited, rendering SGD immune to noise explosion.

4. **Batch Normalization (Scale Invariance):**
   $\hat{X} = (X - \mu)\Sigma^{-1/2}$
   BN normalizes the feature space. For a linear model, the output is invariant to the scale of θ: $BN(c\theta^T x) = BN(\theta^T x)$. This mathematically zeroes out the radial gradient, making true parameter identification impossible.

5. **Ridge Regularization Stabilization:**
   $H_{\text{reg}} = X^T X + \lambda I$
   Ridge regression introduces a penalty that shifts the entire spectrum of eigenvalues upward by λ. This protects the minimum curvature, effectively cutting off adaptive step explosion by ensuring a continuous restoring force.

CHAPTER 7: CONCLUSION

The project mathematically and empirically proves that modern optimizers can achieve perfect loss while internal parameters catastrophically diverge from reality—exposing a critical vulnerability in applied machine learning pipelines known as the **"Optimization Mirage."**

1. **Adam & Momentum** suffer from inverse-variance noise amplification and unconstrained velocity drift along the flat valley floors of highly ill-conditioned ($\kappa=10^7$) landscapes.
2. **Vanilla SGD's** simplicity offers inherent geometric safety, proving to be the most robust unregularized optimizer because its steps directly respect the underlying curvature magnitude.
3. **Batch Normalization** fundamentally destroys scale information, permanently preventing parameter recovery.
4. **Geometric Stabilization (Ridge/L2)** provides the ultimate cure by anchoring the Hessian matrix spectrum, restoring the lost curvature that modern optimizers require to maintain directional stability.

Final Takeaway: Loss monitoring alone is dangerously misleading. Robust system identification requires structure-aware diagnostics and geometric stabilization, not just aggressive optimization.

CHAPTER 8: REFERENCE

1. Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods.
2. Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems.
3. Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization.
4. Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.
5. Santurkar, S., Tsipras, D., Ilyas, A., & Madry, A. (2018). How Does Batch Normalization Help Optimization?
6. Loshchilov, I., & Hutter, F. (2017). Decoupled Weight Decay Regularization (AdamW).